



Mawlana Bhashani Science and Technology University

Project Proposal

Department of Computer Science and Engineering

Project Title: Research and Thesis Repository System (RTRS)

Course Name: Software Development Project-II & Industrial Tour

Course Code: CSE-3116

Date of Submission: 28th February 2026

Submitted By:	Submitted To:
Name: Sumayea Akter Any ID: CE23046	A S M Delowar Hossain,
Name: Mst. Joba Sarker ID: CE23051	Associate Professor,
Session: 2022 – 23	Dept. of Computer Science and Engineering,
3rd Year 1st Semester	MBSTU

Signature of the Supervisor

Contents

1. Project Description	3
1.1 Introduction	3
1.2 Project Overview	3
1.3 Background	3
1.4 Target Market	4
1.5 Possible Risks	4
2. Project Functionality.....	5
2.1 User Authentication and Accounts	5
2.2 Thesis/Project Submission and Upload.....	5
2.3 Search and Filtering	5
2.4 Dashboard and Statistics.....	6
2.5 Supervisor Review and Approval	6
2.6 Functional Summary and Expected Outcomes	6
3. Technical.....	7
3.1 System Architecture.....	7
3.2 Technologies	7
3.2.1 Java 11.....	7
3.2.2 Java Servlet API 4.0	7
3.2.3 Apache Tomcat / Jetty 10	7
3.2.3 MySQL 8.0	8
3.2.4 HTML5, CSS3 and Vanilla JavaScript	8
3.2.5 Google Gson.....	8
3.2.6 Apache Maven	8
3.3 Hardware and Software Requirements	8
4. Project Breakdown	9
4.1 Breakdown	9
4.2 Delivery Stages.....	9
4.3 References	10
Conclusion	10

1. Project Description

1.1 Introduction

The proposed project aims to design and implement a software product for a university academic environment where thesis and project works are submitted, reviewed, archived, and retrieved through a unified digital platform. The central objective is to provide useful, measurable functionality that directly supports students, supervisors, and departmental administrators in managing the lifecycle of academic project documents. In many institutions, the current process remains partially manual, fragmented across personal storage devices, and difficult to monitor, which results in repeated work, delayed review cycles, and poor institutional record keeping.

This proposal introduces TPRS (Thesis and Project Repository System), a structured web-based solution that addresses these gaps by digitizing core activities including submission intake, supervisor review, metadata organization, and searchable repository access. The system is intended to improve transparency, accessibility, and long-term preservation of academic outputs. From a software engineering perspective, the project also serves as a practical implementation of database-driven web application development using Java Servlets, RESTful APIs, and MySQL-backed storage.

1.2 Project Overview

TPRS is a web-based repository system that enables university students to submit, store, and manage thesis and project documents digitally through a role-based interface. Students can upload project files (PDF), provide structured metadata such as title, keywords, department, session, semester, and supervisor, and track the progress of each submission through approval states. Teachers and supervisors receive a dedicated review interface from which they can inspect submission details, approve or reject documents, and maintain a record of supervised works.

A major feature of TPRS is the centralized searchable repository that stores academic works across departments, batches/sessions, and degree levels including Bachelor, Master, and Doctoral programs. The repository is designed to replace the traditional scattered approach where documents are kept in separate departmental computers, personal devices, or printed archives. By consolidating both file content and descriptive metadata, the platform supports faster retrieval, better academic continuity, and more consistent documentation practices within the university.

1.3 Background

At present, many universities in Bangladesh do not maintain a unified digital repository for undergraduate and postgraduate thesis/project records. As a result, students often face difficulty locating previous works for literature support, topic selection guidance, implementation reference, or formatting examples. In practice, access to previous submissions usually depends on informal channels such as senior students, personal messaging groups, or individual faculty collections, which is unreliable and incomplete.

Supervisors and academic coordinators also experience operational inefficiencies because there is no standardized system to track submission status, file versions, or approval history. Without a central platform, monitoring progress across multiple students becomes time-consuming and error-prone.

1.4 Target Market

The primary target market for TPRS is the university academic sector, with an initial deployment focus on the CSE/ICT departments at Maulana Bhashani Science and Technology University (MBSTU). Within this setting, the direct users are students who submit thesis/project documents, teachers and supervisors who review academic work, and departmental administrators who need repository oversight and reporting support.

Although the first implementation is designed around MBSTU requirements, the underlying workflow and data model are generic enough to be adapted for other institutions. The system can be configured for different departments, program structures, and degree levels without changing its fundamental architecture. This makes TPRS a suitable template for broader institutional adoption in Bangladesh where digital thesis management remains underdeveloped.

- Students: submit project/thesis files, manage metadata, and track approval status.
- Teachers/Supervisors: review pending submissions, approve/reject entries, and monitor supervised projects.
- Department Administrators: maintain repository quality, observe repository statistics, and support departmental records.
- University Management: leverage consolidated academic archives for accreditation, reporting, and knowledge preservation.

1.5 Possible Risks

The current prototype assumptions reveal several implementation and operational risks that must be addressed during development and deployment. Identifying these risks early is important for defining mitigation strategies and ensuring that the final software remains secure, reliable, and maintainable in a university environment.

Risk	Potential Impact	Proposed Mitigation
Passwords stored in plaintext	High security risk; exposure of student/teacher credentials if database is leaked.	Implement salted password hashing (e.g., BCrypt) and secure authentication policies.
Single database connection (no pooling)	Reduced performance and connection bottlenecks under concurrent usage.	Use a connection pool (HikariCP/DBCP) and configure resource limits.
Large file upload scalability (50MB PDF limit)	Slow uploads, timeouts, and storage growth issues as repository size increases.	Chunked uploads, server-side validation, and structured file storage strategy.
Dependency on local MySQL server	Reduced availability and portability if the server machine fails.	Deploy to managed or dedicated server with monitored database hosting.
No HTTPS enforcement	Credentials and session cookies may be exposed during transmission.	Enable HTTPS/TLS and secure cookie flags in production.
No backup/restore strategy	Risk of data loss due to server failure, corruption, or accidental deletion.	Automated scheduled backups and periodic recovery testing.

2. Project Functionality

2.1 User Authentication and Accounts

TPRS will support role-based user management with separate account flows for students and teachers. During registration, users provide university ID, full name, email address, department, and phone number, after which the system stores profile data and associates an application role. During login, the system validates credentials and creates a server-side session to maintain authenticated access across pages and API requests.

Role-based access control is a core requirement because students and teachers perform different operations. Students are authorized to create and manage their own submissions, while teachers/supervisors are authorized to review submissions, update workflow status, and access supervision-related listings. The separation of responsibilities reduces misuse and supports cleaner backend authorization logic.

2.2 Thesis/Project Submission and Upload

Students will be able to submit thesis, project, or research documents through a structured form that captures both file data and descriptive metadata. Required fields include title, description/abstract, category (thesis/project/research), keywords, department, academic year, semester, session information, and supervisor selection. A drag-and-drop PDF upload component improves usability and reduces form friction for large document uploads.

Each submission will enter an approval workflow managed by a status field. The typical lifecycle is: pending to in progress to approved/rejected/completed. This workflow enables tracking of supervisory decisions and makes it possible to display transparent progress information to students and administrators. The repository stores the uploaded PDF alongside searchable metadata so approved entries can later be discovered through the search interface.

- **File format:** PDF only (initial implementation, up to 50MB).
- **Metadata validation:** required fields, department consistency, and duplicate prevention checks.
- **Supervisor assignment:** selected from teacher accounts registered in the system.
- **Workflow status:** maintained by supervisor actions and reflected in student dashboard.

2.3 Search and Filtering

TPRS includes a search engine for browsing academic works across departments and sessions. The repository supports full-text search over project title, description, and keyword fields, allowing users to locate relevant documents quickly. To improve retrieval precision, the system also provides structured filters for academic session, degree level, author name, supervisor name, and department.

Autocomplete search suggestions are included to improve user experience, particularly for commonly searched topics, author names, and keywords. This feature helps users find matching entries with fewer typing steps and encourages more effective reuse of prior academic knowledge. Search and filtering together form the primary discovery mechanism of the repository.

2.4 Dashboard and Statistics

The application dashboard provides a summarized view of repository activity and content volume. At a minimum, the dashboard displays total counts of theses, projects, and authors, giving users and administrators a quick understanding of repository growth. Additional panels provide departmental and yearly statistics that support academic reporting and trend analysis.

A recent submissions feed improves visibility of newly uploaded works, while a popular keywords section highlights dominant research themes. These statistics are not only informative for browsing but also useful for department-level decision making, such as identifying active subject areas or underrepresented domains. The dashboard will be implemented using backend aggregated queries and delivered through REST endpoints for efficient frontend rendering.

2.5 Supervisor Review and Approval

Teachers and supervisors are provided with a dedicated review interface that lists pending submissions awaiting action. For each item, supervisors can inspect metadata, open the submitted PDF, and mark the submission as approved or rejected. This workflow digitizes a process that is commonly handled through email attachments or offline file exchange and significantly improves traceability.

The supervisor panel also includes a supervision tracking view that groups all projects under a particular teacher. This allows faculty members to monitor progress across multiple students and review status history in one place. The feature is particularly useful during final project periods when the number of submissions increases and manual tracking becomes difficult.

2.6 Functional Summary and Expected Outcomes

From a user perspective, TPRS delivers end-to-end support for academic project management: account creation, supervised submission, approval workflow, repository discovery, and analytics. From an institutional perspective, it creates a durable digital archive of departmental intellectual output. The functional design is intentionally modular so that future versions can add plagiarism checks, versioning, notification services, and role extensions without redesigning the core repository model.

- Improved accessibility of previous theses/projects for students.
- Standardized supervision and approval workflow for faculty members.
- Centralized departmental archive for administrative reporting.
- Foundation for future enhancements such as notifications and plagiarism integration.

3. Technical

3.1 System Architecture

TPRS follows an MVC (Model-View-Controller) design approach combined with the DAO (Data Access Object) pattern to ensure separation of concerns and maintainable code organization. The Model layer represents business entities such as Student, Teacher, Project, Submission, and Dashboard statistics. The Controller layer is implemented through Java Servlets that receive HTTP requests, validate parameters, invoke services, and return JSON responses. The View layer is the frontend interface built with static HTML, CSS, and JavaScript files.

The backend and frontend are deployed as separate components: the frontend is served independently as static assets, while the backend runs on a servlet container (Apache Tomcat or Jetty) and exposes RESTful JSON APIs. Communication occurs over HTTP using JSON payloads, and a CORS filter is enabled to allow cross-origin access during local development or split deployment. DAO classes isolate direct SQL operations, while service classes encapsulate business logic, resulting in cleaner testing and easier future extension.

- Frontend (HTML/CSS/JavaScript) to REST API requests (JSON)
- Servlet Controllers to Service Layer to DAO Layer to MySQL Database
- CORS Filter for cross-origin communication during split frontend/backend hosting
- WAR deployment on Tomcat/Jetty for standardized servlet execution

3.2 Technologies

3.2.1 Java 11

Java 11 is used as the backend programming language because it provides a stable, widely supported platform for enterprise-style web development and database integration. In this project, Java is used with the Servlet API 4.0 to implement authentication, project management, dashboard, and search endpoints. Its object-oriented design also supports a clean layering of controller, service, and DAO classes.

3.2.2 Java Servlet API 4.0

The Java Servlet API 4.0 provides the request/response programming model used to process HTTP communication in the backend. It enables routing of client requests to servlet classes, handling of sessions, multipart file uploads, and generation of JSON responses. The API is suitable for this academic project because it exposes web internals clearly and helps demonstrate backend fundamentals.

3.2.3 Apache Tomcat / Jetty 10

Apache Tomcat or Jetty 10 will be used as the servlet container to run the compiled Java web application. The backend will be packaged as a WAR file and deployed on the container, typically listening on port 8080 during development. These containers are lightweight, well documented, and commonly used for servlet-based applications.

3.2.3 MySQL 8.0

MySQL 8.0 serves as the relational database for storing users, teachers, students, project metadata, workflow statuses, and repository statistics. The design uses foreign key constraints for data integrity, full-text indexing to support search, and views/stored procedures for reusable query logic and reporting operations. MySQL is appropriate for this project due to its maturity, performance, and easy integration with Java applications.

3.2.4 HTML5, CSS3 and Vanilla JavaScript

The frontend interface is built using HTML5, CSS3, and Vanilla JavaScript without a heavy frontend framework. This keeps the project lightweight and easier to understand for a university software engineering course while still supporting responsive pages and dynamic API-driven behavior. The planned UI design includes custom CSS styling with Poppins font, Material Icons, and gradient-based visual themes for a modern look.

3.2.5 Google Gson

Google Gson is used in the Java backend to serialize and deserialize JSON data exchanged between the frontend and REST APIs. It reduces boilerplate code when converting Java objects into response payloads and parsing incoming request data. This improves maintainability and keeps servlet code cleaner.

3.2.6 Apache Maven

Apache Maven is used for build automation and dependency management in the backend module. It manages external libraries such as Gson and the Servlet API, and standardizes compilation, packaging, and WAR generation through a reproducible project structure. Maven also simplifies environment setup for future maintainers.

3.3 Hardware and Software Requirements

The minimum development and deployment requirements for TPRS are intentionally modest so that the system can be built and tested in a standard university lab or personal computer environment. A typical setup includes JDK 11+, Maven 3.x, MySQL 8.0+, and a servlet container such as Apache Tomcat 9.0+ or Jetty 10. On the client side, any modern web browser is sufficient for accessing the frontend interface.

- Processor: Dual-core or better (recommended: Intel i3/Ryzen 3 or above)
- Memory: Minimum 4 GB RAM (8 GB recommended for smoother development)
- Software: JDK 11+, Maven 3.x, MySQL 8.0+, Tomcat 9.0+ or Jetty 10
- Browser: Latest Chrome, Firefox, Edge, or equivalent
- Network: Stable internet connection for Google Fonts and Material Icons CDN loading

For production deployment, additional requirements should include HTTPS/TLS configuration, regular database backup storage, server monitoring, and scalable storage planning for uploaded PDF files. These are not strict course project requirements but are essential for real institutional adoption.

4. Project Breakdown

4.1 Breakdown

The implementation plan is divided into five major development blocks so that each component can be designed, tested, and integrated in a controlled sequence. This staged structure aligns with standard software project management practices and helps ensure that the database, backend APIs, and frontend interfaces evolve consistently.

- **Database Design and Setup:** schema design for users, departments, projects, supervisors, workflow statuses, and file metadata; creation of tables, keys, indexes, views, and stored procedures.
- **Backend API Development:** implementation of authentication, project, search, and dashboard servlets; service layer logic; DAO classes; session handling; validation; and error responses.
- **Frontend UI Development:** development of Login, Signup, Home/Dashboard, Upload, Search, and Supervisor Review pages with responsive layouts and API integration.
- **Integration and Testing:** end-to-end verification of frontend-backend communication, file upload process, search filters, session management, and approval workflow states.
- **Documentation:** technical documentation, user guide, setup instructions, test results, and final project report preparation.

4.2 Delivery Stages

To maintain progress visibility, the project will be delivered in six incremental stages. Each stage produces a concrete output that can be demonstrated and verified before the next stage begins. This approach reduces risk and supports easier debugging.

Stage	Deliverable	Key Outputs
Stage 1	Database schema and stored procedures	ER design, tables, constraints, views, full-text indexes, stored procedures
Stage 2	Backend REST API with authentication	Auth servlets, session management, DAO + Service layers, JSON responses
Stage 3	Frontend pages with API integration	Login/Signup/Home/Dashboard/Upload UI connected to backend
Stage 4	File upload functionality	Multipart upload, PDF validation, storage path handling, metadata binding
Stage 5	Search, filters, and dashboard statistics	Full-text search, filtering endpoints, repository stats panels
Stage 6	Testing and deployment	Integration testing, bug fixing, WAR packaging, deployment notes

4.3 References

The following references will guide implementation decisions and ensure that the system follows standard technologies and practices used in Java web development and academic repository design.

1. Oracle. Jakarta Servlet Specification (Servlet API 4.0 / related Java EE documentation). Official specification and API guidelines for HTTP servlet-based web applications.
2. Apache Software Foundation. Apache Tomcat Documentation. Deployment and servlet container configuration guidance for WAR-based Java web applications.
3. Eclipse Foundation. Jetty 10 Documentation. Lightweight servlet container documentation and server configuration references.
4. Oracle / MySQL. MySQL 8.0 Reference Manual. Official documentation for schema design, foreign keys, indexing, full-text search, and stored procedures.
5. Apache Maven Project. Maven Documentation. Build lifecycle, dependency management, and packaging workflow for Java projects.
6. Google. Gson User Guide and API Documentation. JSON serialization/deserialization usage for Java applications.
7. DSpace and EPrints documentation (conceptual reference). Examples of institutional repository practices and metadata-driven academic archive systems.
8. Selected university digital repository portals (conceptual review). Used for interface inspiration, search/filter behavior, and repository metadata structuring.

Conclusion

TPRS is proposed as a practical and high-value software solution for improving thesis and project management in university departments. By combining a structured submission workflow, searchable repository access, supervisor review tools, and dashboard reporting, the system addresses a clear institutional gap commonly seen in Bangladeshi universities. The proposed architecture and staged delivery plan make the project feasible within an academic semester while also establishing a strong foundation for future expansion and production-grade improvements.